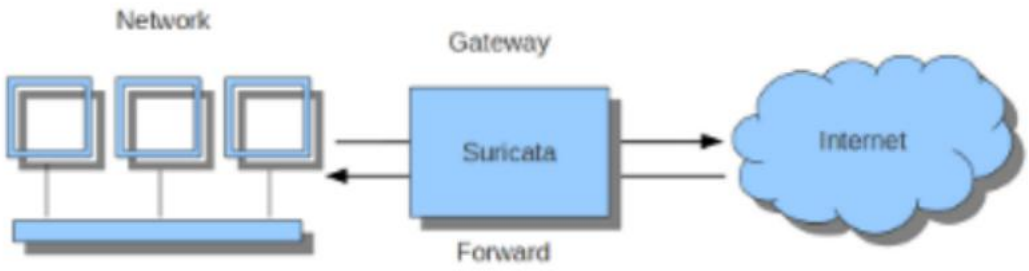
Network Intrusion Prevention using Suricata

Mohamed Amin AYED

Configuring Suricata as inline IPS



we need to set up Suricata as a virtually inline IPS, so we need to pass all inbound and outbound traffic to and from the machine through Suricata. To do so, we are going to use iptables' NFQUEUE;

Instead of immediately letting the packet through or destroying it, NFQUEUE delegates the decision to a **userspace application,** by copying the packet to a numbered queue to which Suricata is listening, inspects and tells kernel what to do.

Which is configured as follows;

```
nfq:
   mode: accept
#  repeat-mark: 1
#  repeat-mask: 1
#  bypass-mark: 1
#  bypass-mask: 1
#  route-queue: 2
#  batchcount: 20
#  fail-open: yes
```

On Suricata

```
amin@ubuntu-vm:~$ sudo iptables -A OUTPUT -j NFQUEUE
amin@ubuntu-vm:~$ sudo iptables -A INPUT -j NFQUEUE
amin@ubuntu-vm:~$
```

On Iptables (queue 0 by dflt)

The Mode indicates the behavior of suricata when handling a packet after it didn't check any prevention boxes, and it could be set as:

- **Accept**; which gives the clean packets a special tag to bypass further iptables rules
- **Repeat;** allows the packet to be subject to further ipitables rules
- **Route;** routes the packet to another queue, for other tools to apply deeper inspection for example

Check Suricata NFQUEUE support:

```
amin@ubuntu-vm:~$ suricata --build-info | grep -i "NFQueue support"
  NFQueue support:                         yes
```

Check the default rules and the default policy for the chains INPUT and OUTPUT

```
amin@ubuntu-vm:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
amin@ubuntu-vm:~$ _
```

→ For now, no rules are set for any of the chains.

```
GNU nano 6.2                          /etc/suricata/suricata.yaml

 # Enable inline operation.  When enabled traffic arriving on a given port is
 # automatically forwarded out its peer port after analysis by Suricata.
 #
 inline: yes
```

This will force Suricata to do stream reassembly in an IPS aware way, unlike the IDS way, prone to evasion.

Preventing against ICMP echo requests

```
amin@ubuntu-vm:~$ cat /var/lib/suricata/rules/custom.rules
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0; itype:8; msg:"Ping Detected & Blocked"; classtype:i
cmp-event; sid:1000001; rev:1;)
```

Custom icmp blocking rule

As shown below, the ping to 8.8.8.8 is blocked whereas to any other IP isn't:

```
amin@ubuntu-vm:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1020ms

amin@ubuntu-vm:~$ ping 8.8.4.4
PING 8.8.4.4 (8.8.4.4) 56(84) bytes of data.
64 bytes from 8.8.4.4: icmp_seq=1 ttl=118 time=227 ms
64 bytes from 8.8.4.4: icmp_seq=2 ttl=118 time=184 ms
```

```
amin@ubuntu-vm:~$ tail -f /var/log/suricata/fast.log
10/27/2026-21:36:35.005735  [wDrop] [**] [1:1000001:1] Ping Dete
cted & Blocked [**] [Classification: Generic ICMP event] [Priori
ty: 3] {ICMP} 192.168.1.147:8 -> 8.8.8.8:0
10/27/2026-21:36:36.007022  [wDrop] [**] [1:1000001:1] Ping Dete
cted & Blocked [**] [Classification: Generic ICMP event] [Priori
ty: 3] {ICMP} 192.168.1.147:8 -> 8.8.8.8:0
10/27/2026-21:36:37.007751  [wDrop] [**] [1:1000001:1] Ping Dete
cted & Blocked [**] [Classification: Generic ICMP event] [Priori
ty: 3] {ICMP} 192.168.1.147:8 -> 8.8.8.8:0
```

This wasn't straight forward as I had to troubleshoot and fix things like; Rule file location. Home IP, interface name, systemctl stop, and start with queue, # mode : accept...

Filtering clear HTTP traffic

```
amin@ubuntu-vm:~$ tail -n 1 /var/lib/suricata/rules/custom.rules
drop tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Dropped HTTP traffic"; classtype:policy-violation; priority:1; sid:1000002;
 rev:1;)
```

Custom http blocking rule

Blocked:

```
amin@ubuntu-vm:~$ curl http://portquiz.com
^C
amin@ubuntu-vm:~$
```

```
d HTTP traffic [**] [Classification: Potential Corporate Pri
vacy Violation] [Priority: 1] {TCP} 192.168.1.147:59778 -> 5
4.243.117.197:80
```

But blocking http based on port number only isn't enough, as you can see using http over any other port works fine:

```
amin@ubuntu-vm:~$ curl http://portquiz.net:445
<html>
<head>
<title>Outgoing Port Tester</title>
<style type="text/css">
body {
        font-family: sans-serif;
        font-size: 0.9em;
}
```

Let's strengthen the rule:

drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic"; content:"User-Agent:"; http_header; nocase; classtype:policy-violation; priority:1; sid:1000003; rev:1;)

And it successfully blocked the http connection regardless of the port:

```
amin@ubuntu-vm:~$ curl http://portquiz.net:445
^C
amin@ubuntu-vm:~$
amin@ubuntu-vm:~$
```

```
10/27/2026-22:01:35.463300  [Drop] [**] [1:1000003:1] Droppe
d HTTP traffic [**] [Classification: Potential Corporate Pri
vacy Violation] [Priority: 1] {TCP} 192.168.1.147:49026 -> 3
5.180.86.68:445
```

Because instead of looking at the port only, which isn't effective, it checks the "User-Agent" striing in the http_header buffer, which is so typical in http traffic

But this is still not effective enough, as sending an empty user agent (no user agent header) successfully bypassed the rule:

```
amin@ubuntu-vm:~$ curl -H "User-Agent:" http://portquiz.
net
<html>
<head>
<title>Outgoing Port Tester</title>
<style type="text/css">
body {
        font-family: sans-serif;
        font-size: 0.9em;
```

Also this isn't effective against https protected websites like https://google.com because the keywords aren't readable to surricata.

TLS analysis

To block all facebook connections for example, we should use `tls.sni="facebook.com"`

drop tls $HOME_NET any -> $EXTERNAL_NET 443 (msg:"Dropped FB"; tls.sni; content:"facebook.com" classtype:policy-violation; priority:1; sid:1000007; rev:1;)

```
amin@ubuntu-vm:~$ curl https://facebook.com
^C
amin@ubuntu-vm:~$
amin@ubuntu-vm:~$
amin@ubuntu-vm:~$
```

```
10/27/2026-22:28:43.991947  [Drop] [**] [1:10
00007:1] Dropped FB [**] [Classification: Pot
ential Corporate Privacy Violation] [Priority
: 1] {TCP} 192.168.1.147:51620 -> 157.240.196
.35:443
```

Activate Windows
Go to Settings to activate Windows.

Blocked

Let’s block access to websites with expired ssl certificates wit this rule:

drop tls any any -> any any (msg:"Expired TLS cert blocked"; tls_cert_expired; classtype:policy-violation; priority:1; sid:1000004; rev:1;)

```
amin@ubuntu-vm:~$ curl  https://expired.badssl.com -k
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="shortcut icon" href="/icons/favicon-red.ico"/>
  <link rel="apple-touch-icon" href="/icons/icon-red.png"/>
  <title>expired.badssl.com</title>
```

Before

```
amin@ubuntu-vm:~$ curl  https://expired.badssl.com -k
^C
amin@ubuntu-vm:~$
amin@ubuntu-vm:~$
```

```
10/27/2026-22:32:16.611000  [Drop] [**] [1:1000004:1] Expired
TLS cert blocked [**] [Classification: Potential Corporate Pri
vacy Violation] [Priority: 1] {TCP} 104.154.89.105:443 -> 192.
168.1.147:34270
```

After

This rule relies on the certificate being visible to Suricata. It can detect expired certificates in TLS 1.2 but not in standard TLS 1.3 connections, where the certificate is encrypted

Downloaded files collection

```
- file-store:
    version: 2
    enabled: yes
```

enable file extraction capability

Adding this rule: `alert http any any -> any any (msg:"FILE store all"; filestore; sid:1; rev:1;)`

```
10/28/2026-13:45:57.316969  [**] [1:1:1] FILE store all [**] [Classif
ication: (null)] [Priority: 3] {TCP} 192.168.137.37:4545 -> 192.168.1
37.254:58048
```

File download over http detected and stored

Generating and analyzing alerts in JSON format

```
- eve-log:
    enabled: yes
    filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
    filename: eve.json
```

Making sure eve json as an output mode

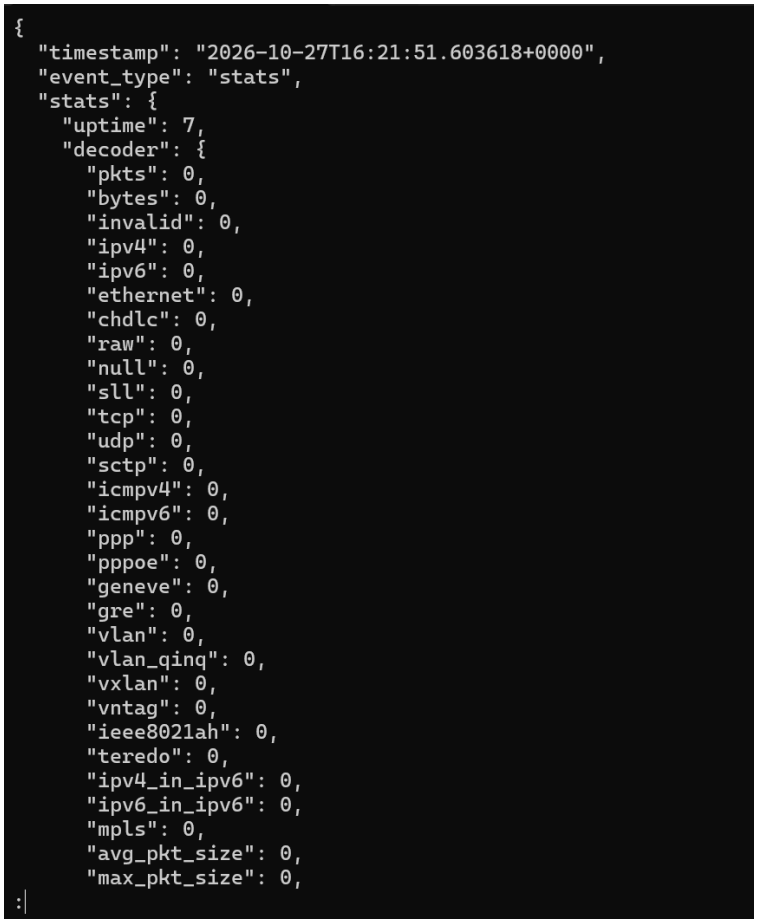
Trying surricata against the WannaCry attack:

```
amin@ubuntu-vm:~$ sudo suricata -r wannacry.pcap -c /etc/suricat
a/suricata.yaml -l /var/log/suricata/ -k none
[sudo] password for amin:
28/10/2026 -- 14:47:35 - <Notice> - This is Suricata version 6.0
.4 RELEASE running in USER mode
28/10/2026 -- 14:48:42 - <Notice> - all 3 packet processing thre
ads, 4 management threads initialized, engine started.
28/10/2026 -- 14:48:43 - <Notice> - Signal Received.  Stopping e
ngine.
28/10/2026 -- 14:48:43 - <Notice> - Pcap-file module read 1 file
s, 46654 packets, 37044839 bytes
amin@ubuntu-vm:~$
```

```
05/18/2017-08:17:17.378605  [**] [1:2200079:2] SURICATA ICMPv6 invali
d checksum [**] [Classification: Generic Protocol Command Decode] [Pr
iority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6992:5661:9d0d:3f96:135 ->
 ff02:0000:0000:0000:0000:0001:ffe3:59ef:0
05/18/2017-08:17:19.597591  [**] [1:2200079:2] SURICATA ICMPv6 invali
d checksum [**] [Classification: Generic Protocol Command Decode] [Pr
iority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6992:5661:9d0d:3f96:135 ->
 ff02:0000:0000:0000:0000:0001:ffe3:59ef:0
05/18/2017-08:17:20.378591  [**] [1:2200079:2] SURICATA ICMPv6 invali
d checksum [**] [Classification: Generic Protocol Command Decode] [Pr
iority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6992:5661:9d0d:3f96:135 ->
 ff02:0000:0000:0000:0000:0001:ffe3:59ef:0
05/18/2017-08:17:20.863131  [**] [1:2200079:2] SURICATA ICMPv6 invali
d checksum [**] [Classification: Generic Protocol Command Decode] [Pr
iority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6992:5661:9d0d:3f96:136 ->
 fe80:0000:0000:0000:f5b3:0851:d70b:06d4:0
```

For example: 133:05/18/2017-08:06:10.905452 [**] [1:2024297:2] ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010 [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.116.149:50069 -> 192.168.116.143:445

Let's check them using `jq`




```
/var/log/suricata$ jq -c 'select(.alert.severity ==1) | [.src_ip, .dest_ip, .alert.signature]' /var/log/suricata/eve.j
```

```
["192.168.1.147","157.240.196.35","Dropped FB"]
["104.154.89.105","192.168.1.147","Expired TLS cert blocked"]
["192.168.116.149","192.168.116.143","ET EXPLOIT ETERNALBLUE Exp
loit M2 MS17-010"]
["192.168.116.149","192.168.116.143","ET EXPLOIT ETERNALBLUE Exp
loit M2 MS17-010"]
["192.168.116.149","192.168.116.138","ET EXPLOIT ETERNALBLUE Exp
```

```
amin@ubuntu-vm:/var/log/suricata$ jq -c 'select(.alert.severity ==1) | [.src_ip, .dest_ip, .alert.signature]' /var/log/suricata/eve.j
son | sort | uniq -c | sort -rn | more
      5 ["192.168.1.147","54.243.117.197","Dropped HTTP traffic"]
      4 ["192.168.116.149","192.168.116.143","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
      4 ["192.168.1.147","91.189.92.22","Dropped HTTP traffic"]
      4 ["192.168.1.147","91.189.91.83","Dropped HTTP traffic"]
      4 ["192.168.1.147","91.189.91.81","Dropped HTTP traffic"]
      3 ["192.168.116.149","192.168.116.172","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
      3 ["192.168.1.147","91.189.92.24","Dropped HTTP traffic"]
      3 ["192.168.1.147","91.189.92.23","Dropped HTTP traffic"]
      3 ["192.168.1.147","91.189.91.82","Dropped HTTP traffic"]
      3 ["192.168.1.147","185.125.190.83","Dropped HTTP traffic"]
      3 ["192.168.1.147","185.125.190.82","Dropped HTTP traffic"]
      3 ["192.168.1.147","185.125.190.81","Dropped HTTP traffic"]
      2 ["192.168.1.147","35.180.86.68","Dropped HTTP traffic"]
      1 ["192.168.116.149","192.168.116.138","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
      1 ["192.168.116.138","192.168.116.172","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
      1 ["192.168.116.138","192.168.116.149","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
      1 ["192.168.1.147","157.240.196.35","Dropped FB"]
      1 ["104.154.89.105","192.168.1.147","Expired TLS cert blocked"]
amin@ubuntu-vm:/var/log/suricata$
```

```
amin@ubuntu-vm:/var/log/suricata$ grep '"event_type":"alert"' /var/log/suricata/eve.json |jq '"\(.timestamp) | \(.alert.gid):\(.alert
.signature_id):\(.alert.rev) | \(.alert.signature) | \(.alert.category) | \(.src_ip):\(.src_port) -> \(.dest_ip):\(.dest_port)"'
"2026-10-27T21:36:35.005735+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:36:36.007022+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:36:37.007751+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:36:38.008704+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:44:37.633647+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:44:38.653557+0000 | 1:1000001:1 | Ping Detected & Blocked | Generic ICMP event | 192.168.1.147:0 -> 8.8.8.8:0"
"2026-10-27T21:52:43.461681+0000 | 1:1000002:1 | Dropped HTTP traffic | Potential Corporate Privacy Violation | 192.168.1.147:59778 -
> 54.243.117.197:80"
```

to determine the detection rule responsible for the following alert generation :

```
"2017-05-18T13:42:07.220702+0530 | 1:2025992:2 | ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags) | A Network Trojan was detected |
192.168.116.149:49368 -> 192.168.116.138:445"
```

```
amin@ubuntu-vm:/var/log/suricata$ grep -i "ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)" /var/lib/suricata/rules/su
ricata.rules
alert smb any any -> $HOME_NET any (msg:"ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)"; flow:to_server,established;
 content:"|ff|SMB|25 00 00 00 00|"; offset:4; depth:9; content:"|00 00 00 00 00 00 00 00 00 00 00 00|"; distance:5; within:10; content:"|23
 00 00 00 07 00 5c 50 49 50 45 5c 00|"; fast_pattern; endswith; threshold: type limit, track by_src, count 1, seconds 30; reference:u
rl,github.com/rapid7/metasploit-framework/blob/master/modules/auxiliary/scanner/smb/smb_ms17_010.rb; classtype:trojan-activity; sid:2
025992; rev:2; metadata:affected_product Windows_XP_Vista_7_8_10_Server_32_64_Bit, attack_target Client_Endpoint, created_at 2018_08_
15, deployment Perimeter, malware_family ETERNALBLUE, confidence Medium, signature_severity Major, tag Description_Generated_By_Proof
point_Nexus, updated_at 2019_09_28;)
```

THE END